CUSTOMER SUPPORT CHATBOT USING DEEP LEARNING AND NATURAL LANGUAGE PROCESSING

A Project Report

Submitted by:
NIKITA
UID: 25MDA10064

in partial fulfillment for the award of the degree of

MASTER OF BUSINESS ADMINISTRATION
IN
(DATA SCIENCE AND ARTIFICIAL INTELLIGENCE)



Chandigarh University
Academic Year 2026–2027

ABSTRACT

Customer support teams receive messages about orders, payments, account access, delivery, returns, and other service issues. Manually reviewing and routing every message can be time-consuming. This project presents an academic prototype of a customer-support chatbot that uses Natural Language Processing (NLP) and Deep Learning to identify the intent behind a customer's message and return a suitable predefined response.

The project uses the supplied Bitext Sample Customer Service training, validation, and testing datasets. The `utterance` column is the input text and `intent` is the target label. The `category` and `tags` columns support exploratory analysis. A TensorFlow/Keras neural network is designed with text vectorization, an embedding layer, global average pooling, a dense hidden layer, dropout, and a softmax output layer. Training, validation, and testing data are used for their separate purposes.

This report documents the problem, dataset, methodology, implementation, evaluation approach, limitations, and future scope. Actual performance values are deliberately left as placeholders until the notebook has been executed. The prototype classifies intent and selects a response from a manually maintained response dictionary; it does not connect to real order, payment, refund, or account-management systems.

Keywords: Customer Support, Chatbot, NLP, Deep Learning, Intent Classification, TensorFlow, Keras.

CONTENTS

CHAPTER 1 — INTRODUCTION

1.1 Background and Motivation

1.2 Problem Statement

1.3 Project Objectives

1.4 Scope of the Project

1.5 Significance

CHAPTER 2 — DATASET AND METHODOLOGY

2.1 Dataset Source and Description

2.2 Input Features and Target

2.3 Data Preprocessing

2.4 Exploratory Data Analysis

2.5 Model Architecture

2.6 Training Strategy

2.7 Evaluation Metrics

CHAPTER 3 — IMPLEMENTATION

3.1 Tools and Technologies

3.2 Project Workflow

3.3 Dataset Loading

3.4 Model Construction and Training

3.5 Prediction and Responses

3.6 Saving and Deployment

3.7 GitHub Repository

3.7 GitHub Repository

The source code and project files are available in the GitHub repository below:

https://github.com/Nikitasingh-81/Customer-support-chatbot

CHAPTER 4 — RESULTS AND DISCUSSION

4.1 Experimental Setup

4.2 Training Behaviour

4.3 Test Evaluation

4.4 Confusion Matrix and Error Analysis

4.5 Chatbot Behaviour

4.6 Limitations

CHAPTER 5 — CONCLUSION AND FUTURE SCOPE

5.1 Conclusion

5.2 Future Scope

REFERENCES

CHAPTER 1

INTRODUCTION

1.1 Background and Motivation

Customer service is important to an organisation's relationship with its customers. Customers may contact a business about orders, payments, returns, account access, delivery, or other service issues. When message volume is high, manually reading and routing each message may increase response time and workload.

A chatbot is a software system that communicates with users through text or speech. This project receives a text message and predicts its intent—the purpose or request expressed by the customer. NLP provides methods for processing human language, while Deep Learning allows a model to learn patterns between messages and intent labels.

The project demonstrates a practical NLP workflow: examining text data, preparing labels, converting text into numerical form, training a neural network, evaluating predictions, and presenting a response through an interactive interface.

1.2 Problem Statement

Customer messages can express similar requests using different words and sentence structures. A keyword-based system may fail when wording changes or a message is ambiguous. This project investigates a deep-learning-based intent classifier that maps customer messages to intent classes and selects a suitable predefined response.

1.3 Project Objectives

- Inspect the supplied customer-service datasets and identify data-quality issues.
- Explore utterances, intents, categories, and tags through descriptive statistics and plots.
- Prepare text and encode intent labels for supervised learning.
- Build a multiclass text-classification model using TensorFlow/Keras.
- Use the training, validation, and testing partitions appropriately.
- Evaluate performance using accuracy, precision, recall, F1-score, and a confusion matrix.
- Analyse incorrect predictions and identify possible sources of error.
- Create a basic chatbot that returns a predefined response for a predicted intent.
- Document limitations and possible future improvements.

1.4 Scope of the Project

The project covers text-based intent classification using the supplied Bitext sample datasets. It includes data inspection, preprocessing, neural-network training, test evaluation, a response-mapping layer, and an optional Streamlit interface. It is an academic prototype: it does not independently generate unrestricted answers or perform real customer-account actions.

1.5 Significance

The project provides practical experience with tokenization, vocabulary construction, embeddings, multiclass classification, regularisation, early stopping, and model evaluation. In a real organisation, a validated intent classifier could help route requests or support automated workflows, subject to privacy safeguards and integration with approved business systems.

CHAPTER 2

DATASET AND METHODOLOGY

2.1 Dataset Source and Description

The project uses three supplied CSV files from the Bitext Sample Customer Service dataset. Their intended partitions are preserved rather than combining them and randomly splitting them again.

Dataset file	**Purpose**	**Approximate raw records***
Bitext_Sample_Customer_Service_Training_Dataset.csv	Fit the model parameters.	6,539
Bitext_Sample_Customer_Service_Validation_Dataset.csv	Monitor training and guide model selection.	818
Bitext_Sample_Customer_Service_Testing_Dataset.csv	Final evaluation on held-out examples.	818

*Counts are preliminary and must be confirmed by running the final data-loading and cleaning code. Report raw and final counts if cleaning removes rows.

Column	**Description**	**Role**
utterance	Customer message in natural language.	Input text (X).
intent	Label representing the purpose of the message.	Target class (y).
category	Broader customer-service category.	Exploratory analysis.
tags	Additional dataset annotations.	Exploratory analysis after inspecting values.

2.2 Input Features and Target Variable

The model uses `utterance` as input and `intent` as target. Intent classification is supervised multiclass classification: each training message has a known label, and the model learns to predict a label for a new message. `category` and `tags` are not automatically treated as model inputs.

2.3 Data Preprocessing

Inspect missing values, empty messages, duplicates, inconsistent label spelling, and unexpected data types before training. Text cleaning should be conservative because punctuation and word choice can carry meaning. A minimal approach is to normalise whitespace and explicitly handle missing utterances or intent labels.

The same intent-to-number mapping must be used for all partitions. The text vocabulary is adapted on training utterances only. Validation and test text are transformed using the learned vocabulary

without adapting it again. Exact or near-duplicate utterances across partitions should be checked and documented; records should not be silently moved.

2.4 Exploratory Data Analysis

EDA describes the data before modelling. The analysis should include dataset dimensions, column names and data types, missing values, duplicate counts, number of unique intents, intent and category distributions, sample utterances, consistency of labels across partitions, and potential cross-partition duplicates.

A bar chart of intent counts can reveal class imbalance. Overall accuracy alone may hide weak performance on minority classes. All charts included in the submitted report should be generated from the actual datasets and clearly labelled.

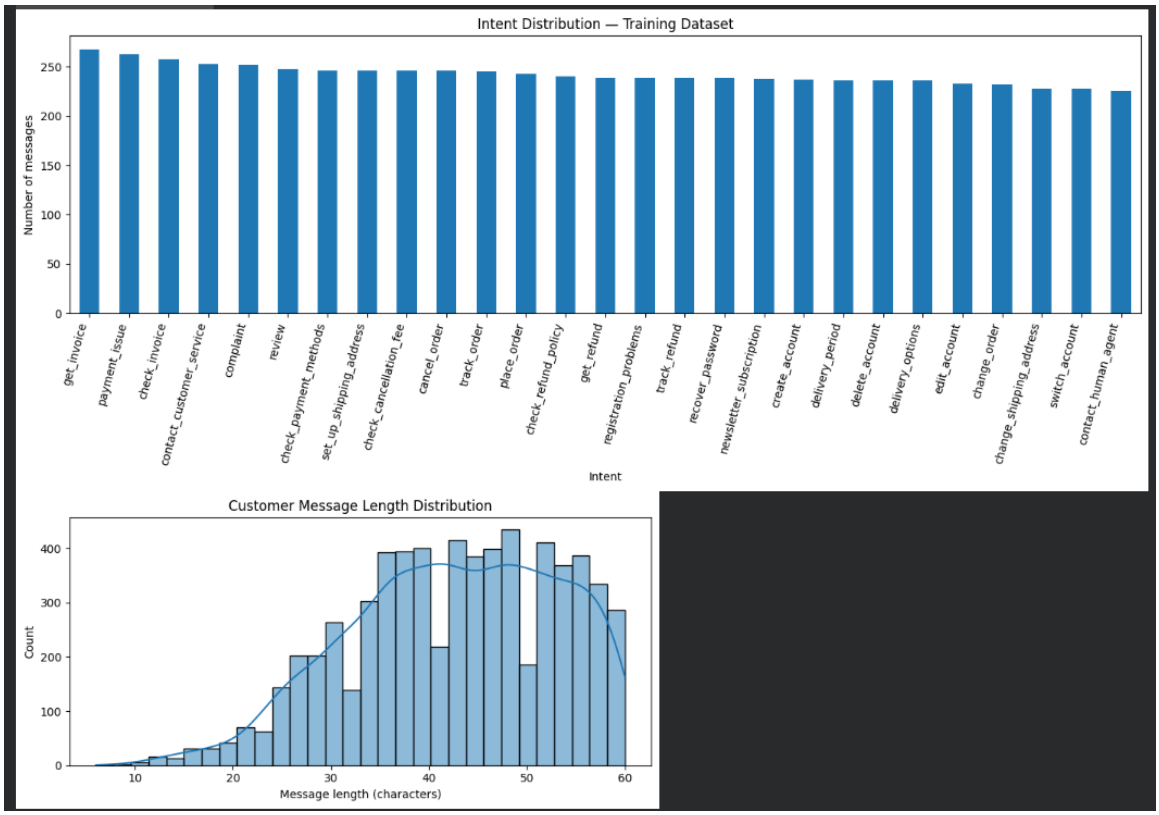


Figure 2.1: Training-set intent distribution and customer-message length distribution.

Interpretation: The displayed intent counts are broadly similar in height, suggesting reasonably comparable sample counts across the intents shown. The message-length histogram indicates that many customer messages fall in the middle-to-upper part of the displayed character range, while very short messages are less frequent. These are visual observations; exact counts and summary statistics should be confirmed from the CSV files.

2.5 Model Architecture

The proposed model is a neural network for multiclass intent classification. TextVectorization converts text to integer token sequences. Embedding learns a dense vector for each token. GlobalAveragePooling1D combines token vectors into a fixed-length message representation. A Dense layer with ReLU learns non-linear patterns, Dropout helps reduce overfitting, and a final Dense layer with softmax produces probabilities for the intent classes.

Component	**Purpose**
TextVectorization	Builds vocabulary and converts text to integer sequences.
Embedding	Learns numerical representations of tokens.
GlobalAveragePooling1D	Combines token vectors into one message-level vector.
Dense (ReLU)	Learns non-linear patterns useful for classification.
Dropout	Randomly disables some activations during training to help reduce overfitting.
Dense (Softmax)	Outputs a probability for each intent class.

This is a relatively simple baseline architecture. It does not explicitly model long-range word order like an RNN or Transformer. Its effectiveness must be established through held-out evaluation.

2.6 Training Strategy

The model is compiled with the Adam optimiser and sparse categorical cross-entropy loss. Adam updates model weights using gradients. The loss measures the difference between predicted probabilities and correct labels. Sparse categorical cross-entropy is suitable for integer-encoded target classes.

Training proceeds in batches over multiple epochs. Validation loss and accuracy monitor learning. Early stopping can stop training when validation loss no longer improves, while model checkpointing can save the best model.

2.7 Evaluation Metrics

Metric	**Meaning**
Accuracy	Fraction of all evaluated messages classified correctly.
Precision	Among messages predicted as one intent, fraction truly belonging to that intent.
Recall	Among messages truly belonging to one intent, fraction correctly identified.
F1-score	Harmonic mean of precision and recall.
Confusion matrix	Shows which actual intents are predicted as which classes.
Test loss	Loss-function value on the held-out test partition.

Macro-average scores give each intent equal weight; weighted scores account for the number of examples in each class. Reporting both can be useful when classes are imbalanced.

CHAPTER 3

IMPLEMENTATION

3.1 Tools and Technologies

Tool	Purpose
Python	Main programming language.
Pandas / NumPy	Data loading, cleaning, and numerical operations.
Matplotlib / Seaborn	EDA and model-performance visualisations.
Scikit-learn	Label encoding and evaluation metrics.
TensorFlow / Keras	Neural-network construction, training, evaluation, and saving.
Google Colab / Jupyter	Interactive development and experiment recording.
Streamlit (optional)	Browser-based chatbot interface.

3.2 Project Workflow

1. Load the three CSV files separately.
2. Inspect schema, missing values, duplicates, and label distributions.
3. Clean utterances conservatively and document any removed records.
4. Encode intent labels consistently across all partitions.
5. Adapt the text vectorizer on training text only.
6. Build and compile the neural network.
7. Train on the training partition and monitor validation performance.
8. Evaluate the selected model on the held-out test partition.
9. Analyse misclassified examples and class-wise metrics.
10. Map predicted intents to reviewed responses and test the interaction.
11. Save the model and metadata; optionally connect them to Streamlit.

3.3 Dataset Loading and Preparation

The following extract illustrates loading the files. In Colab, upload the CSVs or place them in Google Drive and update the paths. Record the actual output after execution.

```
import pandas as pd

train_df = pd.read_csv("Bitext_Sample_Customer_Service_Training_Dataset.csv")
val_df = pd.read_csv("Bitext_Sample_Customer_Service_Validation_Dataset.csv")
test_df = pd.read_csv("Bitext_Sample_Customer_Service_Testing_Dataset.csv")

required = {"utterance", "intent", "category", "tags"}
for name, df in [("train", train_df), ("validation", val_df), ("test", test_df)]:
    missing = required - set(df.columns)
    if missing:
```

```
        raise ValueError(f"{name} dataset is missing columns: {missing}")
    print(name, df.shape)
    print(df.isna().sum())
```

The required-column check prevents later steps from silently running on an unexpected file format. The printed shapes and missing-value counts should be recorded in the final report.

3.4 Model Construction and Training

The model is built using the training partition. Confirm the final vocabulary size, sequence length, class count, and hyperparameters against the executed notebook. The extract below illustrates the architecture.

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

text_vectorizer = layers.TextVectorization(
    max_tokens=20000,
    output_mode="int",
    output_sequence_length=50
)
text_vectorizer.adapt(train_text)  # training text only

num_classes = len(label_encoder.classes_)
model = keras.Sequential([
    keras.Input(shape=(), dtype=tf.string),
    text_vectorizer,
    layers.Embedding(input_dim=20000, output_dim=64, mask_zero=True),
    layers.GlobalAveragePooling1D(),
    layers.Dense(64, activation="relu"),
    layers.Dropout(0.30),
    layers.Dense(num_classes, activation="softmax")
])
model.compile(optimizer="adam",
              loss="sparse_categorical_crossentropy",
              metrics=["accuracy"])
```

In the final implementation, `train_text` must contain cleaned training utterances, and the label encoder must be fitted on training labels. Ensure the embedding input dimension is consistent with the vectorizer configuration.

3.5 Training Procedure

```
early_stopping = keras.callbacks.EarlyStopping(
    monitor="val_loss", patience=3, restore_best_weights=True
)
history = model.fit(
```

```
    train_text, y_train,
    validation_data=(val_text, y_val),
    epochs=20, batch_size=32,
    callbacks=[early_stopping], verbose=1
)
```

An epoch is one pass through the training data. Batch size is the number of examples processed before a weight update. `val_loss` is measured on validation data after each epoch. Early stopping ends training if validation loss does not improve for the configured patience period. `verbose=1` displays a progress bar and training metrics.

3.6 Prediction and Response Generation

The model predicts an intent for a new message. The highest softmax probability is used as a confidence score, but it is not a guarantee of correctness. A separate response dictionary maps supported intents to manually written replies. If confidence is below a threshold selected using validation data, a safe fallback should be used.

```
def predict_intent(message, model, label_encoder):
    probabilities = model.predict([message], verbose=0)[0]
    index = int(probabilities.argmax())
    intent = label_encoder.inverse_transform([index])[0]
    confidence = float(probabilities[index])
    return intent, confidence

# Populate this dictionary with reviewed replies for actual intent labels.
response_map = {}

def chatbot_reply(message, threshold=0.60):
    intent, confidence = predict_intent(message, model, label_encoder)
    if confidence < threshold:
        reply = ("I'm not fully sure I understood. Could you please "
                 "rephrase your request or provide more detail?")
    else:
        reply = response_map.get(
            intent,
            "I understand your request, but I do not have a specific response for it yet."
        )
    return intent, confidence, reply
```

The response map is a template and must be populated with actual intent labels and reviewed replies before the system is described as complete. The threshold shown is illustrative and should be checked against validation data. The prototype does not actually cancel orders, process refunds, track shipments, or access customer accounts.

3.7 Saving and Deployment

```
model.save("customer_support_intent_model.keras")
```

```
# Also preserve the label mapping, response dictionary,
# threshold configuration, and required metadata.
```

Save the trained model and all metadata required to reproduce predictions. If Streamlit is used, the interface can display the customer message, predicted intent, confidence, and response. A working interface does not by itself establish model accuracy.

CHAPTER 4

RESULTS AND DISCUSSION

4.1 Experimental Setup

The training partition updates model weights, the validation partition monitors learning and guides model selection, and the held-out test partition estimates final performance on unseen messages. Do not use the test partition to select the best epoch or tune model settings. Report the actual software versions, number of epochs completed, batch size, vocabulary configuration, and random seed from the notebook.

4.2 Training Behaviour

Plot training and validation accuracy and loss from the Keras history. If training accuracy improves while validation loss rises, the model may be overfitting. If both training and validation performance remain poor, the model may be underfitting or the data preparation and settings may need review.

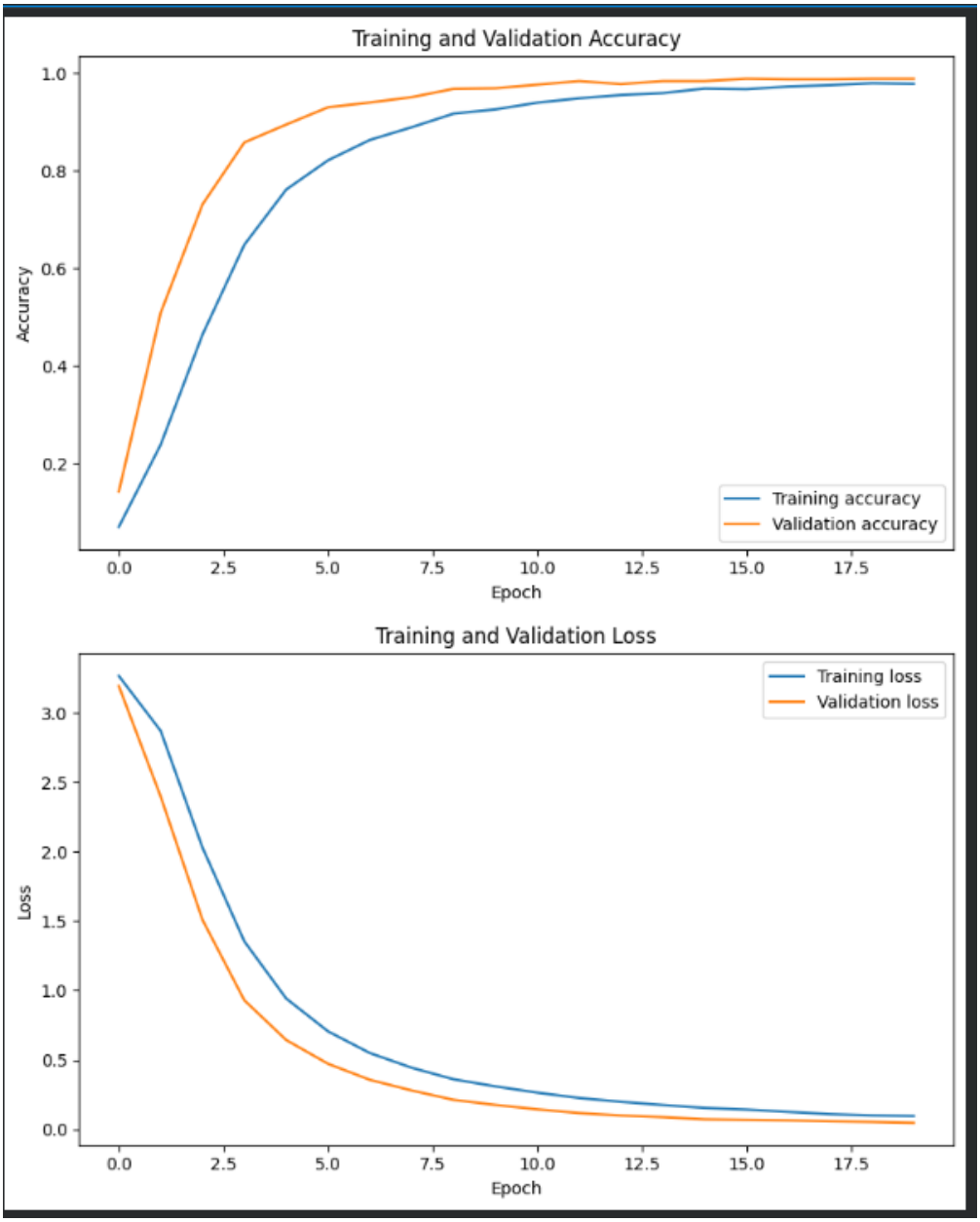


Figure 4.1: Training and validation accuracy and loss across epochs.

Interpretation: In the supplied graph, training and validation accuracy increase over the epochs and approach a high level, while both loss curves decline. The validation loss does not show an obvious late-epoch increase in this plot. Nevertheless, final generalisation performance must be reported using the held-out test metrics.

4.3 Test-Set Evaluation

Enter the measured results after successfully running the final notebook. The supplied plots do not provide verified overall test loss, test accuracy, or macro/weighted precision, recall, and F1-score. Do not infer or invent these values from the figures.

Evaluation measure	**Actual result**
Test loss	[Enter measured value]

Test accuracy	[Enter measured value]
Macro precision	[Enter measured value]
Macro recall	[Enter measured value]
Macro F1-score	[Enter measured value]
Weighted precision	[Enter measured value]
Weighted recall	[Enter measured value]
Weighted F1-score	[Enter measured value]

Accuracy gives the overall fraction of correct predictions. Macro scores highlight per-intent performance equally, while weighted scores reflect class frequency. Interpret the measures together rather than relying on accuracy alone.

4.4 Confusion Matrix and Error Analysis

The confusion matrix compares actual and predicted intents. Values on the main diagonal represent correct predictions; off-diagonal values show confusion between classes. Review misclassified messages with actual intent, predicted intent, and confidence. Possible causes include overlapping intent definitions, ambiguous text, rare phrasing, class imbalance, or inconsistent labels.

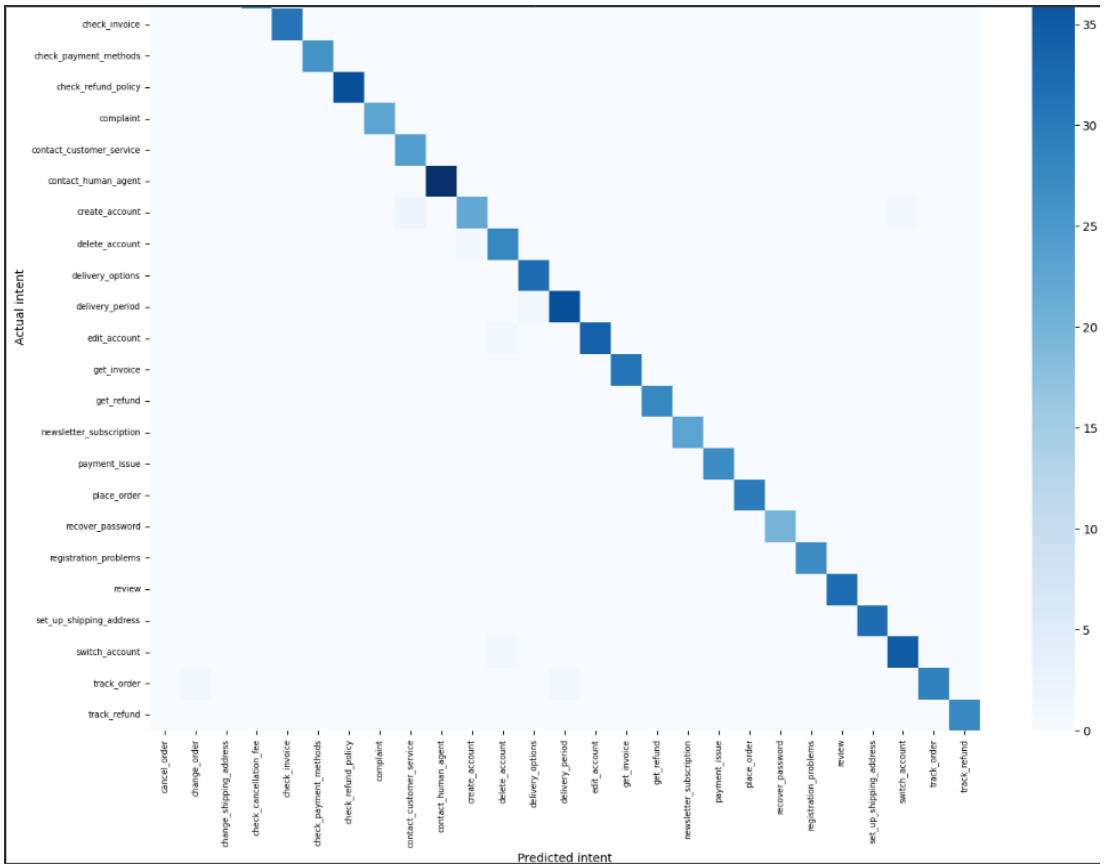


Figure 4.2: Confusion matrix showing actual versus predicted customer-support intents.

Table 4.2: Template for representative test-set misclassifications (complete from actual notebook output).

Customer utterance	Actual intent	Predicted intent	Confidence
[Paste real example]	[Actual label]	[Predicted label]	[0.00–1.00]
[Paste real example]	[Actual label]	[Predicted label]	[0.00–1.00]
[Paste real example]	[Actual label]	[Predicted label]	[0.00–1.00]

4.5 Chatbot Behaviour

The expected interaction is that the user enters a message, the model predicts an intent, the system obtains the top class probability, and the response layer returns the matching predefined reply. Low-confidence messages or intents without a configured reply should receive a fallback response.

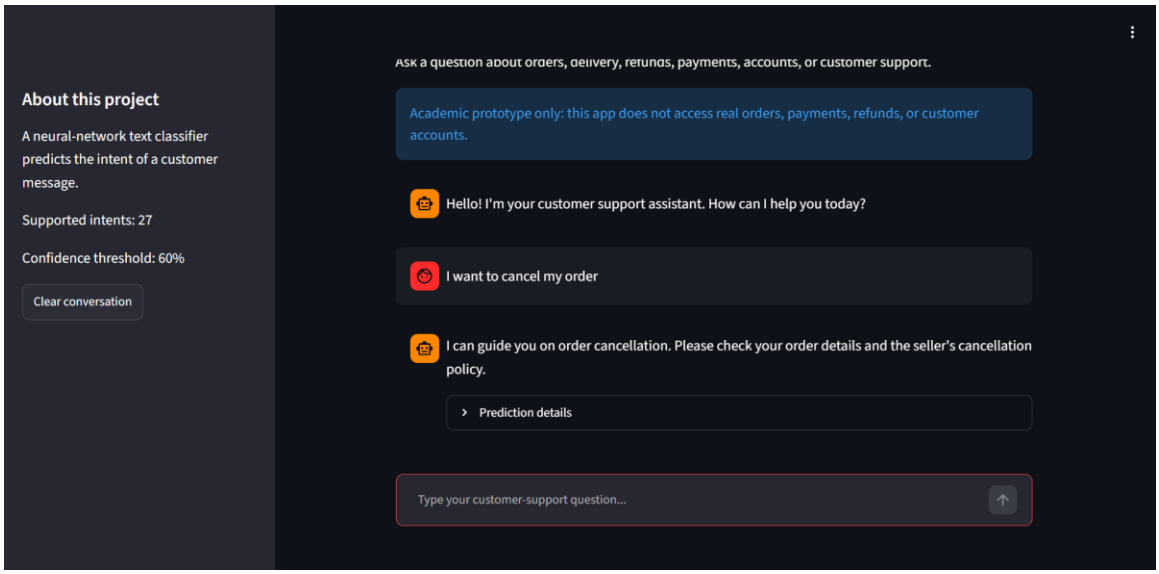


Figure 4.3: Streamlit chatbot interface showing a sample order-cancellation interaction.

The screenshot shows the chatbot title, supported-intent count, configured 60% confidence threshold, welcome message, and message-entry field. It also states that the application is an academic prototype and does not access real orders, payments, refunds, or customer accounts.

Test case	Expected behaviour	Observed result
Clear supported-intent message	Predict intent and return reviewed response.	[Record after testing]
Ambiguous or unfamiliar message	Use fallback when confidence is below validated threshold.	[Record after testing]
Empty message	Ask the user to enter a message.	[Record after testing]
Intent without configured reply	Return safe generic response.	[Record after testing]

Application testing confirms whether the interface starts and the prediction workflow executes. Predictive quality must be assessed separately using test-set metrics.

4.6 Limitations

- The model predicts only among the intent classes represented in its training labels.
- The response dictionary is manually written and does not generate unrestricted answers.
- Confidence scores may not be calibrated and do not guarantee correctness.
- Similar intents and ambiguous messages may cause errors.
- Performance depends on dataset quality, language variety, and class balance.
- The prototype is not connected to real business systems.
- Global average pooling may capture less word-order context than more advanced sequence models.

CHAPTER 5

CONCLUSION AND FUTURE SCOPE

5.1 Conclusion

This project presents an academic prototype of a customer-support chatbot using NLP and Deep Learning. It is designed to classify customer messages into predefined intents using the `utterance` and `intent` fields from the supplied Bitext datasets. The workflow includes dataset inspection, text preparation, label encoding, vectorization, neural-network training, evaluation, error analysis, and response selection.

The proposed Keras architecture uses an embedding layer, global average pooling, a dense hidden layer, dropout, and a softmax output layer. Training, validation, and test partitions are kept separate to support a more reliable evaluation. A response dictionary connects predicted intents to predefined replies.

Final performance conclusions must be written only after the notebook has been executed and the actual metrics and error analysis have been reviewed. This report does not claim an accuracy or F1-score because verified execution results have not been supplied. The prototype demonstrates the core workflow but requires further validation, monitoring, and secure integrations before any production use.

5.2 Future Scope

5.2.1 Improved Language Understanding

Future experiments could compare the baseline with an LSTM, GRU, or pretrained Transformer-based classifier. Comparisons should use the same held-out test set and consider computational cost as well as predictive performance.

5.2.2 Better Confidence Handling

A fallback threshold could be selected using validation data by examining precision, recall, coverage, and the cost of incorrect responses. Calibration techniques could also be investigated.

5.2.3 Expanded and Balanced Data

Additional high-quality examples, especially for underrepresented intents, may improve coverage. Data should be checked for privacy and quality, and evaluation partitions should remain separate.

5.2.4 Knowledge-Base and Business-System Integration

A future version could retrieve approved answers from a maintained knowledge base or connect to order tracking and account workflows through secure APIs. Sensitive actions would require authentication, validation, logging, and human escalation where appropriate. These are future possibilities, not current features.

5.2.5 User Interface and Monitoring

The optional Streamlit interface could add conversation history, user feedback, monitoring dashboards, and escalation options. A deployed system should track unresolved queries, incorrect predictions, response latency, and changes in user language.

REFERENCES

1. Bitext. (n.d.). Sample Customer Service dataset. Supplied training, validation, and testing CSV files.

2. TensorFlow. (n.d.). Text classification with TensorFlow. https://www.tensorflow.org/tutorials/keras/text_classification

3. TensorFlow. (n.d.). TextVectorization API documentation. https://www.tensorflow.org/api_docs/python/tf/keras/layers/TextVectorization

4. TensorFlow. (n.d.). Embedding API documentation. https://www.tensorflow.org/api_docs/python/tf/keras/layers/Embedding

5. Scikit-learn Developers. (n.d.). Classification report. https://scikit-learn.org/stable/modules/generated/sklearn.metrics.classification_report.html

6. Scikit-learn Developers. (n.d.). Confusion matrix. https://scikit-learn.org/stable/modules/generated/sklearn.metrics.confusion_matrix.html

7. Streamlit. (n.d.). Streamlit documentation. https://docs.streamlit.io/